

Applied AI CA1

Samuel Ayers

March 12, 2026

1 Introduction

In this report, we will be implementing a machine learning model to predict diagnosis of depression and/or chronic illness from a medical dataset. The dataset contains demographic, clinical and lifestyle information on 670 patients. We will train several models and compare them using cross-validation and hyperparameter tuning, after selecting the best model we will evaluate the prediction on our test set.

2 Data management

2.1 Problem Modelling

Our target variable here is diagnosis. We have 3 distinct options, Depression, Chronic Illness or Both. Here we can treat this as a multi-class classification problem, i.e. predict diagnosis as either Depression, Chronic Illness or Both, or we can treat it as a multi-label problem. Since Both refers to the patient having both depression and chronic illness, we could instead separately predict whether the patient has depression or chronic illness, which may be more suitable for this problem, since Both is dependent on being diagnosed with Depression and Chronic Illness and is not itself a different class. We will train our models on both formulations of the problem.

2.2 Data Analysis and Preprocessing

We begin by removing the ID column as this is a unique identifier for the patients and serves no purpose in prediction. In addition to this, our data comes entirely from the UK, so we can remove the country column.

Next, we check the data for any null values (none are present in our dataset), outliers or values outside the acceptable range. To check for outliers, we create boxplots for each variable. In our data, we observe a 99, in the `depression_ph9_score`, this is both an outlier and an unacceptable value, so the tuple containing it is removed. From the boxplots, we also observe a large number of outliers for `fatigue_severity_score`, here these are acceptable but rare, so they will not be removed.

Next, to allow our models to process the data, we must encode the categorical variables. We have 3 ordinal categorical variables (`work_status`, `social_activity_level` and `exercise frequency`) which we will encode with label-encoding, and 2 nominal variables (`gender` and `meditation_mindfulness`) since there are only 2 options, we can encode with binary values. For our multi-class models, we will encode the target variable, diagnosis, with label-encoding (i.e. Depression=0, Chronic Illness=1, Both=2) and for our multi-class models we will instead create 2 columns, one for depression and one for chronic illness. In the case where only depression is present, the depression column will have a 1 and chronic illness a zero, and visa versa for when chronic illness is present. When both are present, both columns will feature a 1.

After encoding, we can begin data analysis. Creating a pairplot and correlation matrix, we observe a correlation of 0.97 between `sleep_quality_index` and `deep_sleep_quality_index`, a correlation of this magnitude can negatively effect our model, so we drop the `deep_sleep_quality_index` column.

From the pairplot, we also observe that some variables divide the data based on the target variable well, i.e. (`depression_ph9_score`, `fatigue_serverity_score` and `pem_present`), this may indicate a degree of linear separability, this is also supported by our PCA plot where we see divisions between the classes.

2.3 Experimental Protocol

For training, we split the data into training (80%) and test (20%) sets. We perform hyperparameter tuning using 5-fold cross-validation to ensure robust performance estimates and reduce overfitting. After training, we test our chosen model on the unseen test set.

3 Model Training

Several models were trained for both our multi-class and multi-label problems, a hyperparameter search was conducted using `GridSearchCV` from `scikit-learn`. This function systematically trains the models using 5-fold cross validation on all possible combinations of the given hyperparameters.

Random Forest

A random forest is a collection of decision trees which vote for a label to assign to the data point. Decision trees take a feature vector as input and construct a tree of decisions based on these features. For random forest the following parameters were explored:

- `n_estimators`: Number of trees in the forest [1, 10, 50, 100]
- `max_depth`: Maximum depth of each decision tree [None, 5, 10]
- `max_features`: Number of features considered when splitting nodes [None, 'sqrt', 'log2']

From the search, we observe that we achieve 100% accuracy on most combinations of hyperparameters, that said, when max depth is too low and max features is limited we saw our worst results, albeit > 98% mean test score which is still very high, likely due to a small degree of under-fitting.

K-Nearest Neighbour

K-Nearest neighbour or K-NN is based on the idea of memorising the training and predicting the label of any new instance based on the labels of its K nearest neighbours. For K-NN the following parameters were explored:

- `n_neighbours`: number of neighbours considered (1–20)
- `weights`: Uniform or distance based weighting
- `p`: Distance metric, 1= Manhattan Distance, 2= Euclidean distance

K-NN was our worst performing model achieving a maximum mean test score of 87.85% and 88.04% for the multi-class and multi-label models respectively. Both models performed best with a moderate number of neighbours, distance based weighting and Manhattan distance for the distance metric. The worst results obtained were with only a single neighbour considered and uniform weighting, suggesting giving greater influence to closer neighbours improves accuracy. Here, having the number of neighbours too small leaves the model too prone to noise.

Support Vector Machine

Support vector machines aim to construct a maximum margin separator, which separates the data by creating a decision boundary with the largest possible distance between data points.

- `C`: regularisation parameter [0.1, 1, 10, 100]
- `kernel`: Polynomial or radial basis function (RBF)

- **degree:** Polynomial degree for polynomial kernel

SVM also achieved 100% mean test with our optimal set of hyperparameters. In our tests, the parameter C, which balances maximising the margin and minimising classification error, had a strong effect on our tests. Small values of C yielded very poor results. In addition to this polynomial degree 1 performed best, which is equivalent to a linear kernel. This supports the idea that our data is linearly separable. We also saw no difference between our multi-class and multi-label models here.

Table 1: Summary of Hyperparameter Search Results

Model	Best Params	Best Mean	Best Std	Worst Params	Worst Mean	Worst Std
RandomForest Multiclass	max_depth=None, max_features=None, n_estimators=10	1.0000	0.0000	max_depth=5, max_features=sqrt, n_estimators=10	0.9944	0.0075
RandomForest Multilabel	max_depth=None, max_features=sqrt, n_estimators=10	1.0000	0.0000	max_depth=10, max_features=sqrt, n_estimators=10	0.9944	0.0112
SVM Multiclass	C=100, degree=1, kernel=poly	1.0000	0.0000	C=0.1, degree=4, kernel=poly	0.4785	0.0320
SVM Multilabel	C=100, degree=1, kernel=poly	1.0000	0.0000	C=0.1, degree=4, kernel=poly	0.2187	0.0348
KNN Multiclass	n_neighbors=9, p=1, weights=distance	0.8785	0.0251	n_neighbors=1, p=2, weights=uniform	0.7720	0.0226
KNN Multilabel	n_neighbors=13, p=1, weights=distance	0.8804	0.0415	n_neighbors=1, p=2, weights=uniform	0.8000	0.0192

*Full results in ipynb file

4 Model Evaluation

Although random forest and SVM both achieved perfect cross-validation mean test scores, we will select random forest for our final model. Random forest performed better on more hyperparameter variations, indicating greater robustness. In addition to this, random forest offers better interpretability than SVM which is important in medical contexts.

4.1 Results

Testing our model on the test set achieved the following results:

Table 2: Multi-class random forest evaluation metrics

Class	Precision	Recall	F1-Score	Support
Diagnosis: Depression	1.00	1.00	1.00	54
Diagnosis: Chronic Illness	1.00	1.00	1.00	52
Diagnosis: Both	1.00	1.00	1.00	28
Balanced Accuracy	1.00			

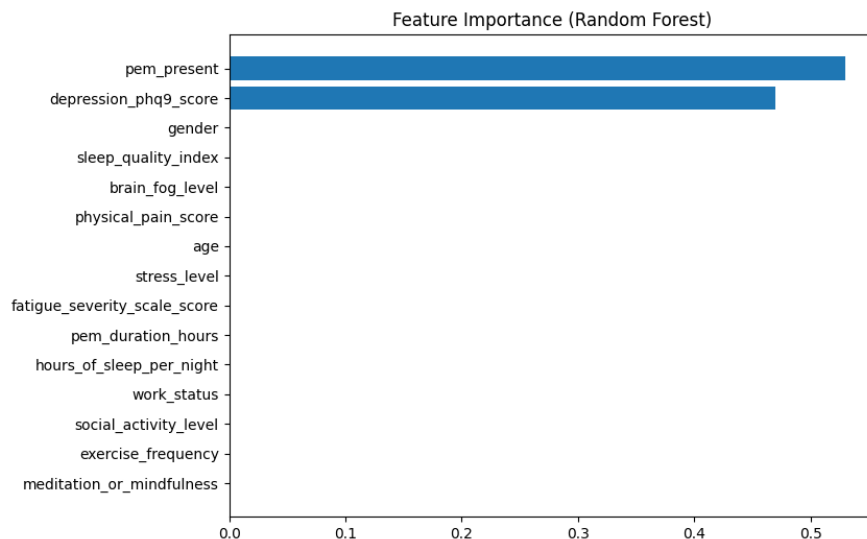
Table 3: Multi-label random forest evaluation metrics

Label	Precision	Recall	F1-Score	Support
Diagnosis: Depression				
Depression not present	1.00	1.00	1.00	52
Depression present	1.00	1.00	1.00	82
Diagnosis: Chronic Illness				
Chronic Illness not present	1.00	1.00	1.00	54
Chronic Illness present	1.00	1.00	1.00	80
Average Balanced Accuracy	1.00			

From our test, we see our random forest model performed perfectly on the test data set, both for multi-class and multi-label. It is unusual to see perfect performance in a machine learning model and is often caused by data leakage, however, this is likely due to the separability of the data.

Due to perfect classification, a confusion matrix is not a useful visual representation here. Instead, we will opt for a feature importance plot. This will indicate what features in our dataset most influence the diagnosis, due to the medical nature of our dataset, this will be a more useful plot that may reveal

interesting information for the client.



From this plot we observe that the model relies solely on depression phq9 score and pem presence to make its predictions. The other features contribute nothing to the model, suggesting that the core information required for prediction lies within these variables. This aligns with our previous prediction from the PCA plot that the data may be highly separable, as is often the case when a few strong features are present.